



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Chapter 5: ARRAYS AND STRINGS

September 2025

Content

-
- | | |
|---|-----------------------------|
| 1 | Compound Types |
| 2 | Arrays |
| 3 | Multidimensional Arrays |
| 4 | Strings |
| 5 | Character Arrays vs Strings |
-

Content

1 Compound Types

2 Arrays

3 Multidimensional Arrays

4 Strings

5 Character Arrays vs Strings

Compound data types

- **Compound data types** are created from basic data types.
- They provide more flexible ways to store and use data.
- Used when basic data types are not sufficient for specific tasks.

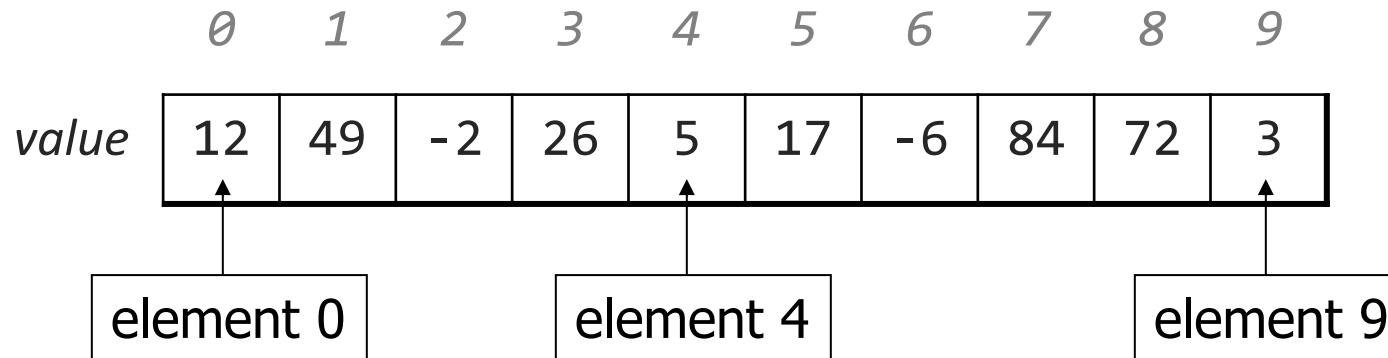
- **Common Types:**
 - Array
 - String
 - Class (or struct)
 - ...

Content

-
- | | |
|----------|-----------------------------|
| 1 | Compound Types |
| 2 | Arrays |
| 3 | Multidimensional Arrays |
| 4 | Strings |
| 5 | Character Arrays vs Strings |
-

Arrays

- An **array** is an **advanced data type** that contains **a set of data** represented by a **single variable name**.
- An **element** is an individual piece of data contained in an array.
- The following figure shows an integer array called `c`.
 - `c[0] = 14; c[1] = 49, c[2] = -2, etc.`



Array Declaration

- The **syntax** for declaring an array is
`<type> <name> [<elements>];`
- Array names follow the same naming conventions as variable names and other identifiers.
- Example:
`int arMyArray[3];`
`char arStuGrade[5];`
- The first declaration tells the compiler to reserve 3 elements for integer array arMyArray.

	0	1	2
value	undefined	undefined	undefined

Subscript

- The numbering of elements within an array **starts** with an index number of **0**.
- An index number is an element's numeric position within an array. It is also called a *subscript*.

`<name>[<index>];`

`<name>[<index>] = <value>;`

arStuGrade	0	1	2	3	4
value	'A'	'B'	'C'	'D'	'F'

- Example:
 - `arStuGrade[0]` refers to the **1st** element in the `arStuGrade` array.
 - `arStuGrade[1]` refers to the **2nd** element in the `arStuGrade` array.
 - `arStuGrade[2]` refers to the **3rd** element in the `arStuGrade` array.
 - `arStuGrade[3]` refers to the **4th** element in the `arStuGrade` array.
 - `arStuGrade[4]` refers to the **5th** element in the `arStuGrade` array.

Example 5.2.1: An example of array

```
#include <iostream>
using namespace std;
int main() {
    char arStuGrade[5] =
        {'A', 'B', 'C', 'D', 'F'};
    for (int i = 0; i < 5; i++)
        cout << arStuGrade[i]
              << endl;
    return 0;
}
```

- The output is:

A
B
C
D
F

arStuGrade	0	1	2	3	4
value	'A'	'B'	'C'	'D'	'F'

Example 5.2.2: Compute the sum of the elements of the array

```
#include <iostream>
using namespace std;
int main() {
    const int arraySize = 12;
    int a[arraySize] = {1, 3, 5, 4, 7, 2, 99, 16, 45, 67, 89, 45};
    int total = 0;
    for (int i = 0; i < arraySize; i++)
        total += a[i];
    cout << "Total of array element values is " << total << endl;
    return 0 ;
}
```

- The output of the above program is as follows :

Total of array element values is 383

Content

1 Compound Types

2 Arrays

3 Multidimensional Arrays

4 Strings

5 Character Arrays vs Strings

Multi-Dimensional Arrays

- C++ allows arrays of any type, including **arrays of arrays**. With **two** bracket pairs we obtain **a two-dimensional array**.
- The idea can be iterated to obtain arrays of higher dimension.

- With each bracket pair we add another dimension.

- Some examples of array declarations:

```
int a[1000];           // a one-dimensional array
int b[4][3];           // a two-dimensional array
int c[7][9][2];        // a three-dimensional array
```

- In these above examples:

- b has 4×3 elements
 - c has $7 \times 9 \times 2$ elements

b[4][3]

[0][1][2]

[0]	1	2	3
[1]	4	5	6
[2]	7	8	9
[3]	10	11	12

A two-dimensional array

- Starting at the base address of the array, all the array elements are stored **contiguously** in memory.
- For the array *b*, we can think of the array elements arranged as follows:

```
int b[4][3];
```

	Column 1	Column 2	Column 3
Row 1	b[0][0]	b[0][1]	b[0][2]
Row 2	b[1][0]	b[1][1]	b[1][2]
Row 3	b[2][0]	b[2][1]	b[2][2]
Row 4	b[3][0]	b[3][1]	b[3][2]

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9
[3]	10	11	12

Example 5.3.1: checks if a matrix is symmetric or not

```
#include <iostream>
using namespace std;
const int N = 3;
int main( )
{
    int i, j;
    int a[N][N];
    bool symmetr = true;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            cin >> a[i][j];
```

a[0][0]	a[0][1]	a[0][2]
a[1][0]	a[1][1]	a[1][2]
a[2][0]	a[2][1]	a[2][2]

```
for(i = 0; i < N; i++) {
    for (j = 0; j < N; j++)
        if(a[i][j] != a[j][i]) {
            symmetr = false;
            break;
        }
    if(!symmetr)
        break;
}
if(symmetr)
    cout << "\nThe matrix is symmetric"
        << endl;
else
    cout << "\nThe matrix is not symmetric"
        << endl;
return 0;
```

if(!symmetr) means if(symmetr == false)

Example 5.3.2

- Write a C++ program to input a list of student IDs and their **three** score columns, namely `score_1`, `score_2`, `score_3` of a course.
- Calculate the **final_score** of each student using following equation:

$$\text{final_score} = 0.2 * \text{score_1} + 0.3 * \text{score_2} + 0.5 * \text{score_3}$$

- Then print out the list of students associated with their scores.

Input:

ID	scores
1	8, 7, 9
2	7, 8, 9
3	9, 8, 9

Output:

ID	scores including final_score
1	8, 7, 9, 8.2
2	7, 8, 9, 8.3
3	9, 8, 9, 8.7

- *Hint: use a matrix of float*

Example 5.3.2

```
#include <iostream>
using namespace std;
const int n = 3;
const int m = 5; // ID, 3 score columns, and the final score
int main() {
    float a[n][m];
    float final_score;
    float rate[3] = {0.2, 0.3, 0.5};

    //input data to the matrix a[n][m] for a score table;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m-1; j++) {
            cout << "Score " << j + 1 << " of student " << i + 1 << ": ";

            cin >> a[i][j];
        }
```


Example 5.3.2 (cond.)

```
//calculate the final_score
for (int i = 0; i < n; i++) {
    final_score = 0;
    for (int j = 1; j < m - 1; j++) // score_1; j = 0 is for ID
        final_score += rate[j-1] * a[i][j];
    a[i][m - 1] = final_score; // update final_score at the last column
}
// Print out the result table (i.e., matrix)
cout << endl << "The student score table is: \n";
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++)
        cout << a[i][j] << " ";
    cout << endl;
}
return 0;
}
```

Content

-
- | | |
|---|-----------------------------|
| 1 | Compound Types |
| 2 | Arrays |
| 3 | Multidimensional Arrays |
| 4 | Strings |
| 5 | Character Arrays vs Strings |
-

String

- Fundamental types represent the most basic types handled by the machines where the code may run. But one of the major strengths of the C++ language is its rich set of **compound types**, of which the fundamental types are mere building blocks.
- An example of **compound type** is the **string** class. Variables of this type are able to **store sequences of characters**, such as words or sentences. A very useful feature!
- A first difference with fundamental data types is that in order to declare and use objects (variables) of this type, the program needs to include the header where the type is defined within the standard library (`#include <string>`)

0 1 2 3 4

H	e	l	l	o
---	---	---	---	---

Example 5.4.1

```
// my first string
#include <iostream>
#include <string>
using namespace std;
int main () {
    string mystring;
    mystring = "This is a string";
    cout << mystring;
    return 0;
}
```

- A variable of string type can be initialized in the variable declaration as follows:
- A variable of string type can be initialized in the second way as follows:

```
string month = "March";
```



```
string s1("Hello");
```

Example 5.4.2

```
#include<iostream>
#include<string>
using namespace std;
int main() {
    string str1; // an empty string
    string str2("Good Morning");
    string str3 = "Hot Dog";
    string str4(str3);
    string str5(str4, 4);
    string str6 = "linear";
    string str7(str6, 3, 3);
    cout << "str1 is: " << str1 << endl;
    cout << "str2 is: " << str2 << endl;
    cout << "str3 is: " << str3 << endl;
    cout << "str4 is: " << str4 << endl;
    cout << "str5 is: " << str5 << endl;
    cout << "str6 is: " << str6 << endl;
    cout << "str7 is: " << str7 << endl;
    return 0;
}
```

- The output of the program:

```
str1 is:
str2 is:  Good Morning
str3 is:  Hot Dog
str4 is:  Hot Dog
str5 is:  Dog
str6 is:  linear
str7 is:  ear
```

Strings Operators

Operator	Description
<code>+=</code>	Appends characters to a string.
<code>=</code>	Assigns new character values to the contents of a string.
<code>[]</code>	Provides a reference to the character with a specified index in a string.
<code>+</code>	Combines (concatenates) two strings.

Example 5.4.3

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string a = "Hello";
    string b = "World";
    a += " ";           // append space into a
    string c = a + b;   // concatenate 2 strings
    cout << c << " | " << c[0] << endl;
    return 0;
}
```

- The output of the program:

Hello World | H

Some methods for string class

- **string objectName(str, n)** : creates and initializes a string object with a substring of string object `str`, starting at index position `n` of `str`.
- **string objectName(str, n, p)** : creates and initializes a string object with a substring of string object `str`, starting at index position `n` of `str` and containing `p` characters.
- **Input a string with `getline()`**
 - The expression `getline(cin, message)` will continuously accept and store characters typed at the terminal until the **Enter** key is pressed. All the characters encountered by `getline()` are stored in the string named `message`.

Example 5.4.4

- The output of the program: typed in by user

Enter a string:

This is a test input of a string

The string just entered is:

This is a test input of a string

```
#include<iostream>
#include<string>
using namespace std;
int main() {
    string message;
    cout << "Ener a string: \n");
    getline(cin, message);
    cout << "The string just entered is:\n")
        << message << endl;
    return 0;
}
```

Some methods for string class (cond.)

- C++ provides two functions to get the number of characters in a string:
 - `length()`
 - `size()`
- **Example 5.4.5:**

```
#include<iostream>
#include<string>
using namespace std;
int main() {
    string str = "This is a string.";
    cout << "Length: " << str.length() << endl;
    cout << "Size: " << str.size() << endl;
    return 0;
}
```

Some methods for string class (cond.)

- The `at()` function returns an indexed character from a string.

- **Example 5.4.6:**

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s = "Hello World!";
    cout << s.at(0) << endl;           // First character ('H')
    cout << s.at(s.length() - 1) << endl; // Last character ('!')
    return 0;
}
```

- The output of the program:

H
!

Content

1 Compound Types

2 Arrays

3 Multidimensional Arrays

4 Strings

5 Character Arrays vs Strings

Character Arrays vs Strings

- Character Arrays

- It is a sequential collection of data type 'char'.
- It doesn't have built-in methods to perform operations on character arrays.
- The characters in a character array can be accessed using index.
- Stored in contiguous memory locations.
- The size of a `char[]` array is fixed and cannot be changed.

- Strings

- It refers to a sequence of characters, which is represented as a single data type.
- It contains built-in functions such as `at()`, `length()`, `size()`.
- Characters can be accessed using index or member functions
- Stored in memory managed automatically.
- A string has a dynamic size that can grow or shrink as needed.

Content

-
- | | |
|---|-----------------------------|
| 1 | Compound Types |
| 2 | Arrays |
| 3 | Multidimensional Arrays |
| 4 | Strings |
| 5 | Character Arrays vs Strings |
-



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Thank you!